

vmn Experiment Tracking

Workflow Guide for Engineers

vmn experiments = snapshot of your code + append-only log of metrics.
No server, no database -- just files on disk (or S3).

Three Workflows Covered:

1. Single Developer -- local laptop, git repo, quick experiments
2. Multi-Developer Team -- shared NFS/FSx mount, concurrent writes
3. K8s at Scale -- hundreds of pods, NFS or S3, zero git

WHAT IS AN EXPERIMENT?

```
+-----+
| Experiment = 3 things: |
| | |
| 1. CODE SNAPSHOT | Your code at that exact moment
| (committed + uncommitted) | (patches, not full copies)
| | |
| 2. METRICS LOG | key=value pairs over time
| (append-only JSONL) | loss=0.34, acc=0.91, ...
| | |
| 3. METADATA | who, when, which branch,
| (YAML) | base version, notes
+-----+
```

Stored in: .vmn/<app>/experiments/<verstr>/

Format: plain files -- YAML + JSONL -- no database

Generated from vmn project documentation

Workflow 1: Single Developer

You're on your laptop. You have a git repo with vmn set up (vmn stamp -r patch my_app). You want to try different configs and track what happened.

```
YOUR LAPTOP
+-----+
| git repo |
| +-----+ |
| | your code (committed+uncommitted) | |
| +-----+ |
|         | |
|         v |
| vmn exp create / vmn exp run |
|         | |
|         v |
| .vmn/my_app/experiments/ |
| +-- 1.0.0-dev.a1b/ |
| | metadata.yml |
| | log.local.jsonl <-- metrics |
| | patches/ <-- diffs |
| +-- 1.0.0-dev.a1b.r2/ |
| (same code = .r2 suffix) |
+-----+
```

Option A: Manual Measurements

Tweak your config, run your test yourself, then record:

```
# Experiment 1: try batch=64
vmn exp create my_app --note "batch=64" \
  --metrics latency_ms=12.3 throughput=8100

# Experiment 2: try batch=128
vmn exp create my_app --note "batch=128" \
  --metrics latency_ms=15.1 throughput=9400

# Compare results
vmn exp list my_app --sort latency_ms
vmn exp diff my_app # code diff + metric delta
```

Each experiment snapshots your entire working tree (committed + uncommitted). If the code hasn't changed between runs, vmn appends .r2, .r3, etc. Nothing is overwritten.

Option B: Let vmn Run Your Command

```
vmn exp run my_app --note "lr=0.01" -- python train.py --lr 0.01
```

What happens step by step:

1. Snapshots your working tree (committed + uncommitted)
2. Launches your command, streams output to your terminal
3. Sets \$VMN_METRICS_FILE -- your script writes key=value lines there
4. vmn tails that file live (metrics appear in the UI in real time)

- When command exits, vmn records exit code + duration

Writing Metrics From Your Script

```
import os

metrics_file = os.environ["VMN_METRICS_FILE"]

def log_metric(key, value, step=None):
    with open(metrics_file, "a") as f:
        prefix = f"step={step} " if step is not None else ""
        f.write(f"{prefix}{key}={value}\n")

for epoch in range(100):
    log_metric("loss", loss, step=epoch)    # -> live training curve
    log_metric("final_acc", acc)           # -> scalar metric
```

step=N gives you training curves in the UI. Without step=, you get scalar metrics.

Works from any language. Shell scripts can do: `echo "loss=0.34" >> $VMN_METRICS_FILE`

Browsing & Comparing

```
vmn exp list my_app           # table of all experiments
vmn exp list my_app --sort loss # sorted by a metric
vmn exp show my_app --latest  # full detail of most recent
vmn exp diff my_app           # code diff + metric delta
vmn exp restore my_app --latest # checkout that code state
vmn ui                        # web dashboard with charts
```

Workflow 2: Multi-Developer Team

Your team shares a repo. Multiple people run experiments at the same time. Everyone wants to see all results in one place.

Key idea: shared NFS mount + per-writer log files = safe concurrent writes

SHARED NFS / FSx MOUNT (/mnt/shared)

```
+-----+
| experiments/my_app/ |
|   +-- 1.0.0-dev.abc.alice/ |
|   |   metadata.yml |
|   |   log.alice-laptop.jsonl |
|   +-- 1.0.0-dev.abc.bob/ |
|   |   metadata.yml |
|   |   log.bob-desktop.jsonl |
|   +-- 1.0.0-dev.def.carol/ |
|   |   metadata.yml |
|   |   log.carol-laptop.jsonl |
+-----+
```

```
      ^           ^           ^
      |           |           |
+-----+ +-----+ +-----+
| Alice | | Bob | | Carol |
| laptop | | desktop | | laptop |
+-----+ +-----+ +-----+
```

Each person: writes their OWN log file (no conflicts)
Reading: vmn merges ALL log.*.jsonl files automatically

Setup: One-Time

Mount a shared filesystem that all team members can access. NFS, AWS FSx, GlusterFS, or any POSIX-compatible shared mount works.

Each Developer's Workflow

```
# Set your writer ID (once, in .bashrc or .zshrc)
export VMN_WRITER_ID=$(hostname) # or your name: alice-laptop

# Run experiments pointing to the shared mount
vmn exp run my_app \
  --experiment-dir /mnt/shared \
  --writer-id $VMN_WRITER_ID \
  --note "lr=0.01, batch=64" \
  -- python train.py --lr 0.01

# Or with manual metrics
vmn exp create my_app \
  --experiment-dir /mnt/shared \
  --metrics loss=0.34 acc=0.91 --note "new optimizer"
```

Why This Is Safe for Concurrent Writes

1. **Writer ID** -- each person gets a unique writer ID. Their metrics go to their own file: log.alice-laptop.jsonl. No file collisions.

2. **Unique verstr** -- each experiment verstr includes the writer ID as suffix: 1.0.0-dev.abc.alice-laptop. No version collisions.
3. **Atomic writes** -- JSONL files use O_APPEND, which is atomic on POSIX for lines under 4KB. No locking needed.

Viewing All Team Results

```
# Anyone on the team can see all experiments:  
vmn exp list my_app --experiment-dir /mnt/shared  
vmn exp diff my_app --experiment-dir /mnt/shared
```

```
# Or launch the web UI for a dashboard + charts:  
vmn ui --repo /mnt/shared
```

vmn merges all log..jsonl files sorted by timestamp -- one unified leaderboard.*

Workflow 3: K8s at Scale

You're running a hyperparameter sweep: 100-1000 pods, each with a different config. No pod has git installed -- they're running from an exported snapshot. Two sub-modes: shared NFS mount (simpler) or S3 (no shared filesystem needed).

Mode A: K8s + Shared NFS / FSx

Recommended when you already have a shared filesystem (EFS, FSx, NFS).

```
CONTROL PLANE (your machine or CI)
+-----+
| 1. vmn snapshot export my_app -o snapshot.tar.gz |
| 2. tar xzf snapshot.tar.gz -C /mnt/fsx/code/      |
| 3. kubectl apply -f sweep-job.yaml                |
+-----+
|
| shared NFS mount: /mnt/fsx
|
+-----+
| Pod 1 | Pod 2 | Pod N |
| lr=0.01 | lr=0.03 | lr=0.1 |
| NO GIT | NO GIT | NO GIT |
| reads: | reads: | reads: |
| metadata | metadata | metadata |
| .yaml | .yaml | .yaml |
| writes: | writes: | writes: |
| log.pod1 | log.pod2 | log.podN |
| .jsonl | .jsonl | .jsonl |
+-----+ +-----+ +-----+
```

Each pod: no git, no locks, no coordination.
 vmn ui --repo /mnt/fsx => leaderboard + live curves

Step 1: Export a Snapshot (Control Plane)

```
# On your machine (has git)
vmn snapshot export my_app --latest -o /mnt/fsx/snapshot.tar.gz
tar xzf /mnt/fsx/snapshot.tar.gz -C /mnt/fsx/code/

# /mnt/fsx/code/ now contains:
#   vmn_metadata.yaml  <- vmn reads this instead of git
#   your_code/         <- the actual source tree
```

Step 2: Each Pod's Entrypoint

```
vmn exp run my_app \
  --from-snapshot /mnt/fsx/code/vmn_metadata.yaml \
  --experiment-dir /mnt/fsx \
  --writer-id $HOSTNAME \
  -- python train.py --lr $LR --batch $BATCH
```

What Each Flag Does

Flag	What it does
--from-snapshot	Reads vmn_metadata.yml instead of git. No git needed.
--experiment-dir	Write to shared mount instead of local .vmn/
--writer-id	Unique pod ID. Metrics -> log.<id>.jsonl (no conflicts)

The experiment verstr includes the pod ID: 1.0.0-dev.abc.def.pod-xyz-123. Zero collision, zero contention, zero coordination between pods.

Step 3: View Results

```
# From your machine or a dashboard server:
vmn ui --repo /mnt/fsx

# CLI quick check:
vmn exp list my_app --experiment-dir /mnt/fsx --sort loss --top 10
```

Mode B: K8s + S3 (No Shared Filesystem)

Use this when pods don't share a filesystem -- each writes locally and syncs to S3.

```

+-----+      +-----+      +-----+
| Pod 1 |      | Pod 2 |      | Pod N |
|       |      |       |      |       |
| /tmp/exp |    | /tmp/exp |    | /tmp/exp |
| log.pod1 |    | log.pod2 |    | log.podN |
| .jsonl  |    | .jsonl  |    | .jsonl  |
+-----+      +-----+      +-----+
|       |      |       |      |       |
| sync   |      | sync   |      | sync   |
| every 30s |    | every 30s |    | every 30s |
| v       |      | v       |      | v       |
+-----+-----+-----+
|                               |
|          S3 BUCKET           |
| s3://my-experiments/vmn-experiments/my_app/ |
|                               |
| +-- 1.0.0-dev.abc.pod1/      |
| |   metadata.yml             |
| |   log.pod1.jsonl (updated every 30s) |
| +-- 1.0.0-dev.abc.pod2/      |
| |   metadata.yml             |
| |   log.pod2.jsonl           |
| +-- 1.0.0-dev.def.podN/      |
| |   metadata.yml             |
| |   log.podN.jsonl           |
+-----+-----+-----+
|
| v
|
vmn ui --s3-bucket my-experiments
(leaderboard + live curves)

```

Step 1: Export Snapshot to S3 or Embed in Image

```

# Option A: Export to S3
vmn snapshot export my_app --latest -o snapshot.tar.gz
aws s3 cp snapshot.tar.gz s3://my-experiments/snapshots/

# Option B: Bake into Docker image (simpler)
# Dockerfile:
# COPY snapshot/ /workspace/
# RUN pip install vmn

```

Step 2: Each Pod's Entrypoint

```

vmn exp run my_app \
  --from-snapshot /workspace/vmn_metadata.yml \
  --experiment-dir /tmp/exp \
  --writer-id $HOSTNAME \
  --backend s3 --bucket my-experiments \
  --sync-interval 30 \
  -- python train.py --lr $LR

```

How S3 Sync Works

TIMELINE OF A SINGLE POD:

```

t=0s      Pod starts. Reads vmn_metadata.yml.
          Creates experiment in /tmp/exp (local).
          Launches: python train.py
          |
t=30s     First sync: uploads log.pod1.jsonl to S3
          (replaces previous S3 object)
          |
t=60s     Second sync: uploads again (more metrics now)
          |
...       Every 30 seconds, same thing
          |
t=end     train.py exits.
          FINAL sync: uploads complete log to S3.
          Records exit code + total duration.
          No metrics lost.

```

- **Near-live:** Metrics appear in the UI within 30 seconds of being written
- **No data loss:** Final sync happens on exit -- nothing is lost even if it crashes mid-interval
- **Any S3:** Uses standard S3 PutObject -- works with AWS, MinIO, LocalStack, R2

Step 3: View Results

```
vmn ui --s3-bucket my-experiments --s3-prefix vmn-experiments
```

```

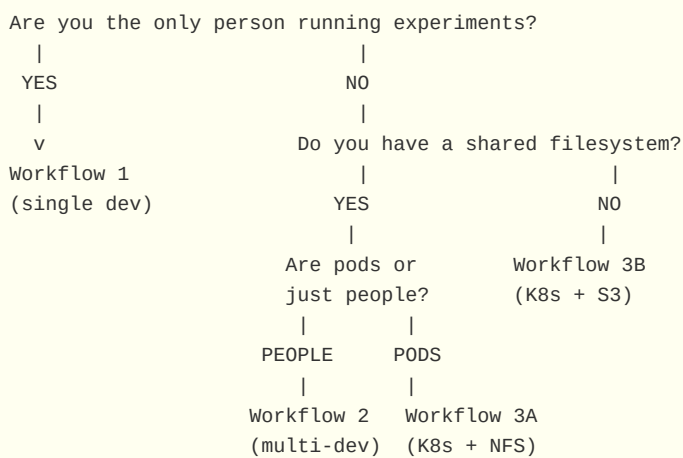
# Same UI as local mode: leaderboard, training curves,
# side-by-side comparison, code diffs

```

Choosing a Workflow

	Single Dev	Multi-Dev NFS	K8s NFS / S3
Git needed?	Yes	Yes	No (snapshot)
Shared FS?	No	Yes (NFS/FSx)	NFS or S3
Concurrent?	No	Yes	Yes (1000+ pods)
Writer ID?	Optional	Recommended	Required
S3 support?	No	No	Yes
Setup	None	Mount + env var	Snapshot export

DECISION TREE:



Reference: Addressing Experiments

Every command that takes an experiment reference supports these forms:

Form	Meaning
(omitted)	The latest experiment
--latest	Most recent, explicitly
@N	N-th row from 'exp list' (1-indexed)
unique prefix	e.g. -v 1.6.0-dev.a1b (shortest unique match)
full verstr	Exact match

Reference: Environment Variables

Variables You Set (pod spec / .bashrc)

Variable	Purpose
VMN_WRITER_ID	Unique writer ID (or use --writer-id flag)
VMN_EXPERIMENT_DIR	Shared mount path (or use --experiment-dir flag)
VMN_SNAPSHOT_METADATA	Path to vmn_metadata.yml (or use --from-snapshot)

Variables Set BY vmn (for your training script)

Variable	Purpose
VMN_EXPERIMENT_ID	The verstr assigned to this run
VMN_APP_NAME	The app name
VMN_METRICS_FILE	Path to write key=value metric lines

Reference: CLI Flags for Multi-User / K8s

Flag	Description
--from-snapshot <path>	Path to vmn_metadata.yml (skips git)
--experiment-dir <path>	Write experiments to shared mount or scratch dir
--writer-id <id>	Unique writer ID for this pod/process
--sync-interval <sec>	Seconds between S3 metric syncs (default: 30)
--backend s3	Use S3 storage backend
--bucket <name>	S3 bucket name
--endpoint-url <url>	Custom S3 endpoint (MinIO, LocalStack)
--prefix <prefix>	Key prefix in bucket (default: vmn-experiments)

Quick Start: K8s Pod Spec (NFS)

```
containers:
- name: experiment
  image: my-training:latest
  command:
    - vmn
    - exp
    - run
    - my_app
    - --from-snapshot
    - /mnt/fsx/code/vmn_metadata.yml
    - --experiment-dir
    - /mnt/fsx
    - --writer-id
    - $(HOSTNAME)
    - --
    - python
    - train.py
  env:
    - name: HOSTNAME
      valueFrom:
        fieldRef:
          fieldPath: metadata.name
  volumeMounts:
    - name: shared
      mountPath: /mnt/fsx
volumes:
- name: shared
  persistentVolumeClaim:
    claimName: fsx-experiments
```

Quick Start: K8s Pod Spec (S3)

```
containers:
  - name: experiment
    image: my-training:latest
    command:
      - vmn
      - exp
      - run
      - my_app
      - --from-snapshot
      - /workspace/vmn_metadata.yml
      - --experiment-dir
      - /tmp/exp
      - --writer-id
      - $(HOSTNAME)
      - --backend
      - s3
      - --bucket
      - my-experiments
      - --sync-interval
      - "30"
      - --
      - python
      - train.py
    env:
      - name: HOSTNAME
        valueFrom:
          fieldRef:
            fieldPath: metadata.name
      - name: AWS_DEFAULT_REGION
        value: us-east-1
# No shared volume needed -- S3 is the shared storage
```

End-to-End Example: Hyperparameter Sweep

Here's the full lifecycle, from setup to viewing results:

1. Stamp a base version (on your laptop, once):

```
vmn stamp -r patch my_app
vmn snapshot export my_app --latest -o snapshot.tar.gz
```

2. Upload to shared storage:

```
# NFS: extract to the mount
tar xzf snapshot.tar.gz -C /mnt/fsx/code/

# S3: upload and bake into image
aws s3 cp snapshot.tar.gz s3://my-experiments/snapshots/
```

3. Launch the sweep:

```
# Your Kubernetes Job template runs vmn exp run per pod  
# (see pod specs above)
```

4. Watch live results:

```
vmn ui --repo /mnt/fsx          # NFS mode  
vmn ui --s3-bucket my-experiments # S3 mode  
  
# Open http://localhost:8265 -- leaderboard + training curves
```

5. Pick the winner:

```
vmn exp list my_app --experiment-dir /mnt/fsx --sort loss --top 5  
vmn exp show my_app -v <best-verstr> --experiment-dir /mnt/fsx  
vmn exp restore my_app -v <best-verstr> # checkout that code
```